Maschinelles Lernen (ML), Wintersemester 2026/27
Arbeitsgruppe Diskrete Systeme

Übung 2: Daten & Explorative Datenanalyse

Bevor ein Lernalgorithmus zum Einsatz kommt, steht in den meisten realen ML-Projekten zunächst die Arbeit mit den Rohdaten: sichten, verstehen, aufbereiten. In dieser Übung lernen Sie mit `pandas` und `matplotlib` die Werkzeuge kennen, mit denen ein Datensatz zuerst *explorativ* untersucht und anschließend in eine für Lernalgorithmen geeignete Form gebracht wird: Aufteilung in Trainings- und Testdaten, Skalierung numerischer und Kodierung kategorialer Merkmale. Dieser Ablauf bildet das Grundgerüst, in das sich die folgenden Übungen (Regression, Klassifikation, Bäume, neuronale Netze) einfügen: Erst werden die Daten verstanden und vorbereitet, dann erst folgt das eigentliche Modell.

Zwei Begriffe werden im Folgenden durchgehend verwendet: Ein *Datenpunkt* (auch *Sample* oder *Beobachtung*) ist ein einzelner Eintrag des Datensatzes, etwa eine vermessene Blüte. Ein *Merkmal* (*feature*) ist eine der Größen, die für jedes Beispiel erfasst wurden, etwa eine gemessene Länge. Die *Zielgröße* schließlich ist diejenige Größe, die ein Modell später vorhersagen soll.

2.1 Explorative Datenanalyse (EDA)

Bevor man ein Modell auf einen Datensatz ansetzt, sollte man ihn verstehen: Wie viele Beispiele und Merkmale liegen vor? Welche Wertebereiche, Verteilungen und offensichtlichen Zusammenhänge gibt es? Gibt es fehlende Werte oder Ausreißer? Dieser Schritt heißt *explorative Datenanalyse* (EDA).

Das Werkzeug dafür ist in dieser Übung die Python-Bibliothek `pandas`. Ihre zentrale Datenstruktur ist der *DataFrame*: eine Tabelle mit benannten Spalten (je Spalte ein Merkmal oder die Zielgröße) und durchnummerierten Zeilen, von denen jede ein einzelnes Beispiel enthält. Ein DataFrame verhält sich damit ähnlich wie ein Tabellenblatt, lässt sich aber programmatisch auswerten. Zwei seiner Methoden geben einen ersten Überblick: `.describe()` liefert statistische Kennzahlen je numerischer Spalte (Mittelwert, Standardabweichung, Minimum, Maximum, Quartile), `.info()` nennt Spaltennamen, Datentypen und die Anzahl der tatsächlich vorhandenen (nicht fehlenden) Werte. Ergänzend liefern Visualisierungen mit `matplotlib` einen Eindruck, der sich aus reinen Zahlentabellen nur schwer erschließt: Histogramme zeigen die Verteilung einzelner Merkmale, Streudiagramme (*scatter plots*) die Zusammenhänge zwischen zwei Merkmalen.

Öffnen und bearbeiten Sie das Notebook `ML_U02_Vorlage.ipynb`.

1. Laden Sie den Iris-Datensatz[1] als `pandas`-DataFrame: Funktion `load_iris` aus dem Modul `sklearn.datasets`, aufgerufen mit dem Argument `as_frame=True`. Der Datensatz enthält je 50 Blüten dreier Schwertlilien-Arten mit vier gemessenen Merkmalen: Länge und Breite des Kelchblatts (*sepal length*, *sepal width*) sowie Länge und Breite des Blütenblatts (*petal length*, *petal width*). Verschaffen Sie sich mit `.describe()` und `.info()` einen ersten Überblick.

[1] Fisher (1936): The Use of Multiple Measurements in Taxonomic Problems.

2. Stellen Sie die Verteilung eines Merkmals als Histogramm dar und erzeugen Sie ein Streudiagramm von Blütenblattlänge gegen Blütenblattbreite (*petal length* vs. *petal width*), wobei die Punkte nach Klassenzugehörigkeit (der Schwertlilien-Art) eingefärbt werden.

3. Berechnen Sie mit `.corr()` die paarweisen Korrelationen zwischen den numerischen Merkmalen. Der dort berechnete Korrelationskoeffizient liegt zwischen -1 und $+1$ und misst, wie eng zwei Merkmale linear zusammenhängen: Werte nahe $+1$ bedeuten, dass das eine Merkmal mit dem anderen annähernd geradlinig wächst, Werte nahe -1 einen ebenso engen gegenläufigen Zusammenhang, Werte nahe 0 keinen linearen Zusammenhang. Welches Merkmalspaar korreliert am stärksten, und passt das zu Ihrem Eindruck aus dem Streudiagramm?

2.2 Vorverarbeitung: Train/Test-Split, Skalierung, Kodierung

Rohdaten liegen nur selten bereits in der Form vor, die ein Lernalgorithmus erwartet. Die folgenden vier Aufbereitungsschritte sind so grundlegend, dass sie in vielen ML-Projekten in dieser oder ähnlicher Form vorkommen.

Train/Test-Split: Um die Güte eines Modells später ehrlich beurteilen zu können, wird der Datensatz vor dem Training in eine Trainingsmenge (zum Anpassen der Modellparameter) und eine Testmenge (zur abschließenden Bewertung, wird während des Trainings nicht angefasst) aufgeteilt, häufig im Verhältnis 70:30 bis 80:20. Ab Übung 5 kommt zusätzlich eine dritte Teilmenge hinzu, die *Validierungsmenge*: Sie wird schon während der Modellentwicklung ausgewertet, um zwischen mehreren Modellvarianten bzw. Hyperparametern auszuwählen (s. Abb. 1). In `scikit-learn` übernimmt die Aufteilung die Funktion `train_test_split`.

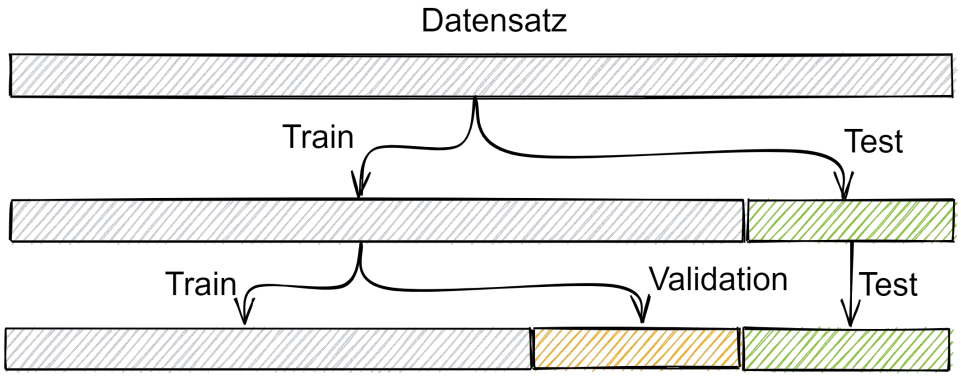


Abbildung 1: Aufteilung eines Datensatzes: der vollständige Datensatz (oben), die Aufteilung in Trainings- und Testmenge im Verhältnis 80:20 (Mitte) sowie die zusätzliche Abtrennung einer Validierungsmenge im Verhältnis 60:20:20 (unten), wie sie ab Übung 5 verwendet wird. Die angegebenen Verhältnisse sind gängige Richtwerte, keine festen Vorgaben.

Skalierung numerischer Merkmale: Viele Algorithmen reagieren empfindlich auf stark unterschiedliche Wertebereiche der Merkmale. Ein Merkmal „Einkommen in Euro“ (Größenordnung 10^4) würde ein Merkmal „Anzahl Kinder“ (Größenordnung 10^0) allein wegen seiner Zahlenwerte dominieren, obwohl beide fachlich gleich wichtig sein können. Der `StandardScaler` zieht dagegen von jedem Wert einer Spalte den Mittelwert dieser Spalte ab und teilt durch deren Standardabweichung; das transformierte Merkmal hat damit Mittelwert 0 und Standardabweichung 1.

Kodierung kategorialer Merkmale: Ein *kategoriales* Merkmal nimmt nur endlich viele Ausprägungen an, die keine Zahlen sind, z. B. „Fahrzeugtyp“ mit den Ausprägungen „PKW“, „LKW“ und „Bus“. Solche Merkmale müssen in Zahlen übersetzt werden, bevor ein Lernalgorithmus mit ihnen rechnen kann. Dabei unterscheidet man zwei Fälle: *Nominale* Merkmale haben keine natürliche Reihenfolge: Zwischen „PKW“ und „Bus“ gibt es kein sinnvolles „kleiner“ oder „größer“. *Ordinale* Merkmale haben eine solche Reihenfolge, z. B. „niedrig“ < „mittel“ < „hoch“, auch wenn die Abstände zwischen den Stufen nicht festgelegt sind.

Beim *One-Hot-Encoding* erhält jede Ausprägung eine eigene Spalte, in der für ein Beispiel eine 1 steht, wenn es diese Ausprägung besitzt, und sonst eine 0. Aus einer Spalte „Fahrzeugtyp“ werden so drei Spalten. Da alle Ausprägungen dadurch gleichrangig behandelt werden, ist dies bei nominalen Merkmalen die übliche Wahl. Beim *Label-Encoding* wird jeder Ausprägung stattdessen eine einzelne ganze Zahl zugeordnet $(0, 1, 2, \ldots)$. Das ist sparsamer, führt aber eine Rangordnung und feste Abstände ein und passt daher vor allem zu ordinalen Merkmalen, bei denen die vergebene Reihenfolge der tatsächlichen entspricht.

Fehlende Werte: Reale Datensätze enthalten oft Einträge, für die kein Wert erfasst wurde; in `pandas` stehen dort Platzhalter vom Typ `NaN` (für *not a number*). Viele Lernalgorithmen können damit nicht umgehen. Der `SimpleImputer` ersetzt solche Platzhalter systematisch durch einen aus den übrigen Werten derselben Spalte berechneten Ersatzwert, z. B. den Mittelwert (bei numerischen Spalten) oder den häufigsten Wert (bei kategorialen Spalten).

Damit diese unterschiedlichen Schritte für numerische und kategoriale Spalten gebündelt und reproduzierbar auf neue Daten angewendet werden können, bietet `scikit-learn` den `ColumnTransformer`: Er wendet je nach Spaltenauswahl unterschiedliche Transformationen an und fasst die Ergebnisse zu einer gemeinsamen Merkmalsmatrix zusammen, also zu einer Tabelle, deren Zeilen die Beispiele und deren Spalten die fertig aufbereiteten, rein numerischen Merkmale sind.

1. Für diese Aufgabe steht im Notebook ein kleiner, künstlich erzeugter Tabellendatensatz („Fahrraddaten“, s. Notebook) mit numerischen Spalten, einer kategorialen Spalte und einigen absichtlich eingefügten fehlenden Werten bereit. Teilen Sie ihn mit `train_test_split` in Trainings- und Testdaten auf.
2. Behandeln Sie die fehlenden Werte mit einem `SimpleImputer`.
3. Skalieren Sie die numerischen Spalten mit einem `StandardScaler` und kodieren Sie die kategoriale Spalte mit einem `OneHotEncoder` (alternativ `pd.get_dummies`). Bündeln Sie beides nach Möglichkeit in einem `ColumnTransformer`.

2.3 Data Leakage

Ein subtiler, aber häufiger Fehler besteht darin, den `StandardScaler` (oder andere Vorverarbeitungsschritte) auf dem *gesamten* Datensatz zu fitten und *danach* erst in Trainings- und Testdaten aufzuteilen. Dabei fließen Mittelwert und Standardabweichung der Testdaten in die Skalierung der Trainingsdaten mit ein. Das Modell hat so, wenn auch nur indirekt und in geringem Umfang, bereits „Informationen“ über die Testdaten gesehen, bevor diese zur ehrlichen Bewertung herangezogen werden. Dieses Phänomen heißt *Data Leakage*.[2] Der korrekte Ablauf ist umgekehrt: erst splitten, dann

[2] Kaufman, Rosset, Perlich (2012): Leakage in Data Mining: Formulation, Detection, and Avoidance.

den Scaler ausschließlich auf den Trainingsdaten fitten und mit den so gelernten Parametern anschließend *sowohl* Trainings- als auch Testdaten transformieren (`fit_transform` auf den Trainingsdaten, nur `transform` auf den Testdaten).

1. Vergleichen Sie im Notebook beide Vorgehensweisen („Scaler zuerst auf allen Daten fitten“ vs. „erst splitten, dann Scaler nur auf Trainingsdaten fitten“) anhand der resultierenden skalierten Trainingsdaten.
2. Auch wenn der numerische Unterschied zwischen beiden Varianten hier klein ausfallen kann: Warum ist das erste Vorgehen dennoch grundsätzlich problematisch, insbesondere bei kleinen Datensätzen oder wenn später die Modellgüte auf den Testdaten berichtet werden soll?